ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ




# BÁO CÁO HRI

BÀI THỰC HÀNH 06

## GIAO TIẾP PHI NGÔN NGỮ ĐIỀU KHIỂN TURTLEBOT 4 BẰNG CỬ CHỈ TAY

**Họ và tên sinh viên:** Đỗ Việt Anh

**Mã sinh viên:** 23020719

**Lớp:** K68E-RE

**Giảng viên hướng dẫn:** Kỹ sư Nguyễn Quốc Bảo

Mã nguồn GitHub: turtlebot4_bai06

Video demo: Google Drive – Bài 06

# Mục lục

**1 Mục tiêu và môi trường sử dụng** **1**
**2 Cấu trúc hệ thống và luồng dữ liệu** **2**
**3 Nhận diện và quy ước sáu cử chỉ** **3**
**4 Điều khiển hành vi và cơ chế dừng** **4**
**5 Kết quả chạy thử** **5**
**6 Cách chạy và liên kết bài nộp** **6**
**Tài liệu tham khảo** **6**

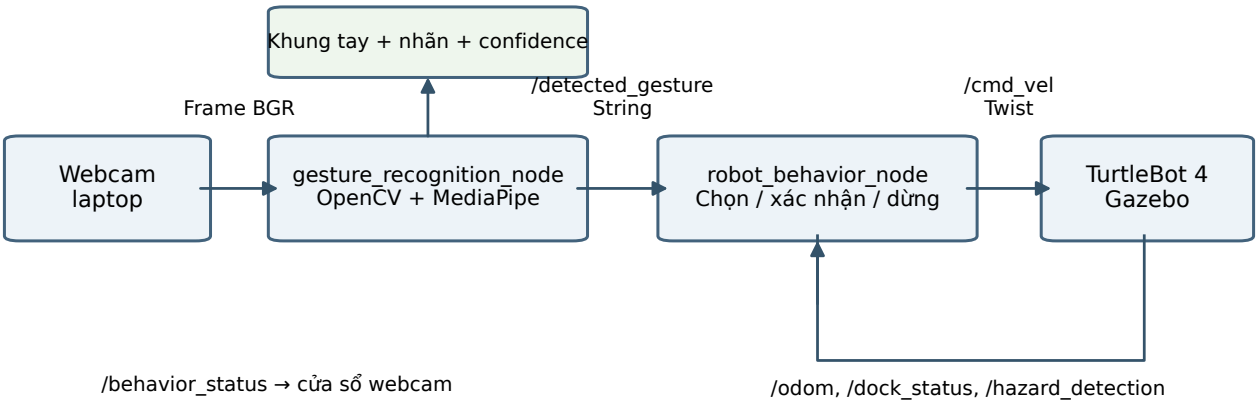
## 1 Mục tiêu và môi trường sử dụng

Bài thực hành sử dụng webcam laptop để nhận diện cử chỉ tay bằng OpenCV và MediaPipe, sau đó điều khiển TurtleBot 4 trong Gazebo qua ROS 2. Người điều khiển chọn hành vi bằng tay, xác nhận bằng ngón cái và có thể dừng hoặc hủy khi cần. Các chuyển động trong phần demo được điều khiển bằng cử chỉ.

| Thành phần | Môi trường sử dụng |
|---|---|
| Hệ điều hành, ROS | Ubuntu 22.04, ROS 2 Humble, Python 3.10.12. |
| Mô phỏng | TurtleBot 4 Standard, Ignition Gazebo Fortress. |
| Nhận diện | MediaPipe Gesture Recognizer 0.10.21, model `gesture_recognizer.task`. |
| Ảnh và hiển thị | OpenCV 4.11.0, NumPy 1.26.4, webcam laptop `/dev/video0`. |
| World | Sàn trống 12 × 12 m; giới hạn điều khiển $\lvert x\rvert, \lvert y\rvert \leq 4{,}5\,\text{m}$. |

**Bảng 1:** *Môi trường triển khai Bài 06.*

## 2 Cấu trúc hệ thống và luồng dữ liệu




**Hình 1:** *Hai node nhận diện và hành vi trao đổi qua topic ROS 2.*

`gesture_recognition_node` dùng OpenCV đọc ảnh, đổi BGR sang RGB và chạy MediaPipe. Node lấy nhãn cùng điểm nhận diện, vẽ 21 điểm và đường nối bàn tay, hiển thị trạng thái robot rồi publish kết quả mới qua `/detected_gesture`.

`robot_behavior_node` subscribe nhãn cử chỉ, lưu lựa chọn và chỉ phát vận tốc sau xác nhận. Timer điều khiển chạy 20 Hz, độc lập với suy luận ảnh để tiếp tục kiểm tra điều kiện dừng khi camera hoặc nhận diện bị gián đoạn.

| Topic | Kiểu message | Vai trò |
|---|---|---|
| `/detected_gesture` | `std_msgs/msg/String` | Nhãn cử chỉ mới, kể cả `NO_HAND`/`UNKNOWN`. |
| `/cmd_vel` | `geometry_msgs/msg/Twist` | Vận tốc thẳng `linear.x` và quay `angular.z`. |
| `/odom` | `nav_msgs/msg/Odometry` | Quãng đường và góc quay thực hiện. |
| `/dock_status` | `irobot_create_msgs/msg/DockStatus` | Chặn chạy khi robot chưa ra khỏi dock. |
| `/hazard_detection` | `irobot_create_msgs/msg/HazardDetectionVector` | Nhận bump, cliff, stall, wheel drop. |
| `/behavior_status` | `std_msgs/msg/String` | JSON trạng thái, hành vi, pha và chu kỳ. |

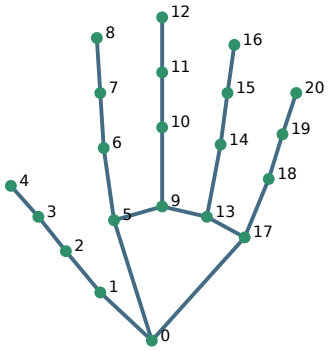
**Bảng 2:** *Các topic chính.*

**Các file trong package.** `gesture_recognition_node.py` xử lý webcam và model; `wave_detector.py` nhận chuyển động vẫy tay; `visualization.py` vẽ khung tay. `robot_behavior_node.py` quản lý hành vi và điều kiện dừng. `config/gestures.yaml` lưu ngưỡng nhận diện, tốc độ và thời gian chờ. `launch/demo.launch.py` mở Gazebo, bridge, robot và hai node; `worlds/gesture_arena.sdf` tạo môi trường trống.

## 3 Nhận diện và quy ước sáu cử chỉ

| Cử chỉ | Cách thực hiện | Hành vi |
|---|---|---|
| Wave | Xòe tay, đưa bàn tay qua lại theo phương ngang. | Chọn chào: xoay trái, phải rồi về hướng gần ban đầu. |
| Pointing Up | Duỗi ngón trỏ lên, gập các ngón còn lại. | Chọn tiến–lùi 4 chu kỳ, tổng 8 chặng. |
| Victory | Duỗi ngón trỏ và ngón giữa thành chữ V. | Chọn đi theo quỹ đạo tròn một vòng. |
| Thumbs Up | Nắm các ngón và giơ ngón cái. | Xác nhận lựa chọn, thực hiện đúng một lần. |
| Open Palm | Xòe cả năm ngón về phía camera. | Dừng ngay khi đang chạy, xóa lựa chọn. |
| Closed Fist | Nắm kín bàn tay. | Hủy hành vi, trở về trạng thái chờ. |

**Bảng 3:** *Quy ước cử chỉ và hành vi. Mọi hành vi chuyển động đều cần xác nhận.*




**Năm cử chỉ tĩnh.** MediaPipe trả về nhãn và confidence. Chương trình dùng ngưỡng 0,60; tên class của ngón cái trong model là `Thumb_Up`. Confidence là điểm do model trả về, không phải độ chính xác của toàn hệ thống.

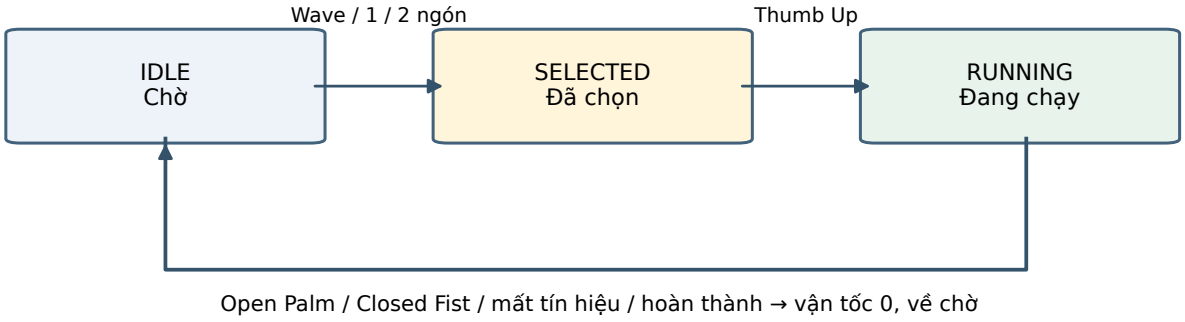
**Cử chỉ Wave.** Model không có class Wave. Chương trình theo dõi cả tâm bàn tay và đầu ngón để nhận vẫy ngang hoặc vẫy bằng cổ tay. Cần hai lần đổi chiều trong 2,4 s; ngưỡng mỗi chặng bằng 30% bề rộng tay, chặn trong 2,5–7% chiều rộng ảnh. Landmarks tay mở vẫn được dùng khi model trả UNKNOWN. Tay đứng yên không tạo Wave; mất tay hoặc kết quả quá cũ xóa chuỗi.

**Hình 2:** *Sơ đồ 21 điểm bàn tay.*

**Phân biệt Wave và Open Palm.** Khi robot đang chạy, Open Palm được chuyển thẳng thành lệnh dừng; không chờ nhận Wave. Khi đứng yên, Wave chỉ tạo lựa chọn chào. Cửa sổ hiển thị bộ đếm 3 s để đổi từ tay mở sang Thumb Up; tay mở trong đoạn này giữ lựa chọn, không tạo lệnh chạy. Hết đoạn chuyển mà vẫn mở tay thì xóa lựa chọn. UNKNOWN ngắn tối đa 0,18 s không xóa các mẫu xác nhận; nó không gia hạn watchdog. Mất tay vẫn dừng ngay.

Cửa sổ webcam hiển thị ảnh gương, khung tay, nhãn, điểm model và trạng thái robot. Chỉ một tay được dùng để điều khiển. Kết quả có nhiều tay được coi là mơ hồ và làm robot dừng. Sau khi xác nhận, cần giữ một cử chỉ hợp lệ trong ảnh để duy trì tín hiệu.

## 4 Điều khiển hành vi và cơ chế dừng




**Hình 3:** *Luồng chọn, xác nhận và thực hiện hành vi.*

**Chào.** Sau Wave rồi Thumb Up, robot quay trái khoảng 0,45 rad, quay phải 0,90 rad, sau đó quay trái 0,45 rad để trở về gần hướng ban đầu. Mỗi pha kết thúc theo góc quay tích lũy từ odometry.

**Tiến–lùi.** Sau Pointing Up rồi Thumb Up, robot tiến 0,40 m và lùi 0,40 m cho mỗi chu kỳ, lặp bốn chu kỳ. Tiến độ được tính bằng độ dời odom chiếu lên hướng bắt đầu chặng. Giao diện hiển thị số chu kỳ từ 1/4 đến 4/4. Trong Gazebo, launch cấu hình Create 3 ở chế độ `safety_override=backup_only` để cho phép lùi hết 0,40 m; node vẫn kiểm tra hazard, vùng chạy và lệnh dừng.

**Đi tròn.** Sau Victory rồi Thumb Up, robot xoay tại chỗ để tâm vòng nằm xa dock, rồi chạy với $v = 0{,}12\,\text{m/s}$, $\omega = 0{,}20\,\text{rad/s}$. Bán kính cấu hình là $R = v/\omega = 0{,}60\,\text{m}$. Robot dừng khi tổng góc quay ở pha đi tròn đạt $2\pi$, không chỉ dựa vào thời gian ước lượng.

| Điều kiện | Xử lý trong chương trình |
|---|---|
| Cử chỉ chọn/xác nhận | Cần ít nhất 3 kết quả cùng nhãn, ổn định tối thiểu 0,2 s. |
| Giữ nguyên cử chỉ | Chỉ tạo một sự kiện; không chạy lại hoặc xếp thêm lệnh trong khi đang chạy. |
| Open Palm, Closed Fist | Publish Twist bằng 0 ngay trong callback; xóa lựa chọn và về IDLE. |
| Mất tay, camera lỗi, nhiều tay | Dừng ngay khi nhận kết quả tương ứng. |
| Không có cử chỉ hợp lệ mới | Watchdog 0,5 s theo monotonic time; UNKNOWN không gia hạn quyền chạy. |
| Kết quả suy luận đến muộn | Bỏ kết quả quá 0,25 s; không dùng nhãn cũ làm tín hiệu sống. |
| Mất odom hoặc có hazard | Odom quá 0,5 s, ra ngoài vùng chạy hoặc bump/cliff/stall/wheel drop đều dừng. |

**Bảng 4:** *Lọc nhiễu, chống lặp và điều kiện dừng.*

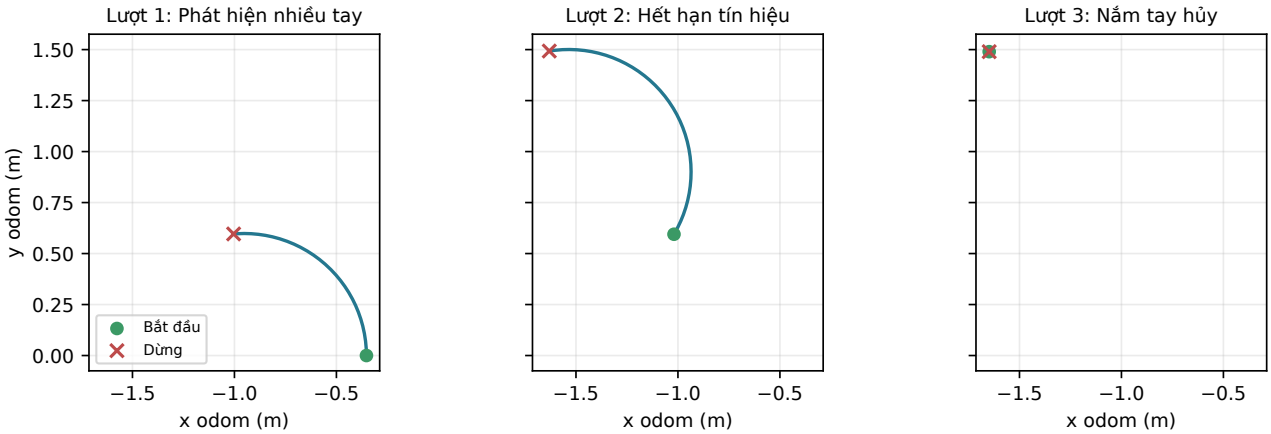
Vận tốc đầu ra bị chặn ở 0,15 m/s và 0,5 rad/s. Sau dừng hoặc mất tín hiệu, robot cần được chọn và xác nhận lại; tín hiệu trở lại không tự tiếp tục hành vi. Các khoảng cách, góc và tốc độ trên là giá trị cấu hình điều khiển.

## 5 Kết quả chạy thử

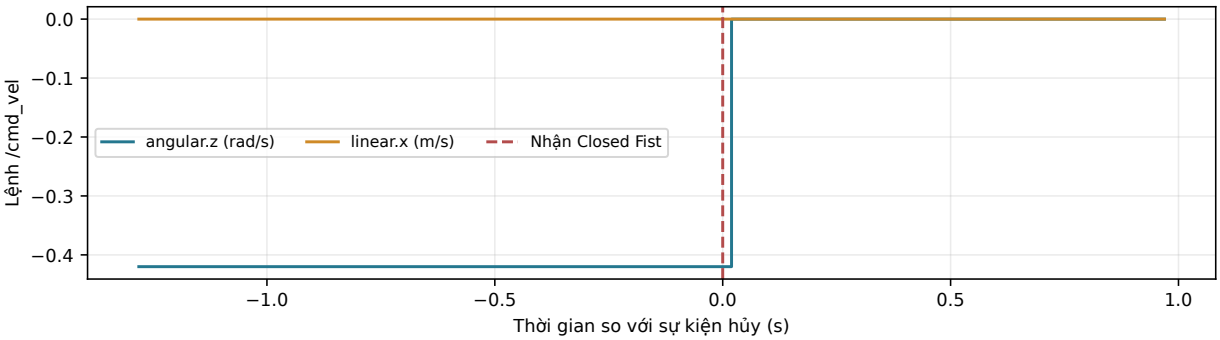
Webcam laptop đã đọc được ảnh thật 640 × 480. Gazebo khởi chạy TurtleBot 4, action `/undock` trả về `SUCCEEDED`, robot về trạng thái chờ. Trong các lượt thử bằng webcam, log đã ghi nhận đủ năm nhãn tĩnh và các sự kiện chọn, xác nhận, hủy, mất tay hoặc hết hạn tín hiệu.

| Nội dung | Kết quả quan sát |
|---|---|
| Chọn và xác nhận | Pointing Up/Victory tạo SELECTED; Thumb Up chuyển sang RUNNING. |
| Đi tròn và căn hướng | Có pha ALIGN_CIRCLE rồi CIRCLE; quỹ đạo odom thay đổi theo lệnh vận tốc. |
| Hủy bằng nắm tay | Closed Fist đưa robot về IDLE, lệnh vận tốc về 0. |
| Mất tín hiệu, nhiều tay | Ghi nhận dừng với GESTURE_TIMEOUT và MULTIPLE_HANDS. |
| Kiểm tra node trong Gazebo | Hoàn thành 4 chu kỳ, 8 chặng; mỗi chặng đo được 0,403–0,405 m, tổng thời gian thực 29,37 s. Lượt kiểm tra dùng nhãn cử chỉ giả, không thay thế demo webcam. |
| Kiểm tra logic | 17/17 trường hợp kiểm tra callback và odom giả qua; gồm xác nhận, chống lặp, 8 chặng và điều kiện dừng. |

**Bảng 5:** *Kết quả có trong dữ liệu chạy thử và kiểm tra logic.*




**Hình 4:** *Quỹ đạo odom ba lượt thử đi tròn, kết thúc do nhiều tay, hết tín hiệu và nắm tay.*




**Hình 5:** *Lệnh vận tốc về 0 sau sự kiện Closed Fist. Đây là lệnh điều khiển, không phải phép đo thời gian thân robot dừng hẳn.*

Các lượt trên dùng để quan sát chạy và dừng giữa chừng. Log hiện có chưa ghi nhận hoàn thành chào, đủ bốn chu kỳ tiến–lùi và một vòng tròn bằng webcam; Wave và Open Palm khi đang chạy cũng cần đối chiếu thêm trong lượt demo đầy đủ.

## 6 Cách chạy và liên kết bài nộp

**Cài đặt.** Máy cần ROS 2 Humble, TurtleBot 4 Simulator và Python 3.10. Trong workspace ROS 2, đặt package ở `src/bai06_turtlebot4`. Các bước cài phụ thuộc và build được ghi trong README của repo.

**Chạy demo trên máy đã cấu hình.**

```
cd /home/admin1/xla/wk4/week6/ros2_ws/src/bai06_turtlebot4
./scripts/run_demo.sh
```

Script mở Gazebo và webcam, căn cửa sổ cạnh nhau, chuẩn bị undock rồi về chờ. Đợi robot ra khỏi dock trước khi chọn cử chỉ. Terminal in thư mục lưu kết quả và ROS domain của phiên chạy. Nhấn Ctrl+C tại terminal đó để đóng phiên.

**Thao tác điều khiển.** Đưa một tay vào ảnh, thực hiện Wave, Pointing Up hoặc Victory để chọn; khi màn hình hiện SELECTED thì giơ Thumb Up để xác nhận. Tiếp tục giữ một cử chỉ hợp lệ trong ảnh trong lúc robot chạy. Xòe tay để dừng, nắm tay để hủy. Không dùng bàn phím để phát lệnh chuyển động trong phần demo.

**Mã nguồn và video.**

- GitHub: github.com/vanhhhhh512/turtlebot4_bai06.
- Video demo: thư mục Google Drive Bài 06.
- Báo cáo PDF và source LaTeX: thư mục `docs/report/` trong repo.

## Tài liệu tham khảo


1. Google AI Edge, *Gesture recognition task guide.* `https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer`
2. Google AI Edge, *Gesture recognition guide for Python.* `https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer/python`
3. OpenCV, *Video I/O.* `https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html`
4. TurtleBot 4, *Simulation.* `https://turtlebot.github.io/turtlebot4-user-manual/software/simulation.html`